Assignment 0 - Group 5

Financial Engineering: Risk Management and Buy side

Gabriele Alippi – Matricola 308871
Elisa Colombo – Matricola 299422
Simone Colombo – Matricola 315162
Giacomo Costa – Matricola 315085

1 Introduction

The goal of this assignment is to bootstrap the discount factor curve for the Euribor 3M interbank market as of the 15th of February 2008, using a single-curve model. The bootstrap procedure combines three types of market instruments — deposits, STIR futures, and interest rate swaps — each covering a different segment of the curve, and produces as output both discount factors $B(t_0, t_i)$ and continuously compounded zero rates $z(t_i)$, evaluated at the settlement date and at the expiry dates of the quoted underlyings.

The implementation relies on auxiliary functions provided in `date_functions.py`:

- `year_frac_act_x`: computes the year fraction under the ACT/x convention.
- `year_frac_30e_360`: computes the year fraction under the 30E/360 convention.
- `business_date_offset`: shifts dates and adjusts them to the nearest business day.

2 Market Data

The pricing date is the 15th of February 2008, with settlement date t_0 = 19th February 2008. Market data consists of three sets of instruments: deposits, STIR futures, and interest rate swaps. Blue rows indicate the instruments included in the bootstrap.

2.1 Deposits

Tenor	Maturity	BID (%)	ASK (%)	Mid (%)
sn	2008-02-20	3.990	4.030	4.010
1w	2008-02-26	4.050	4.090	4.070
1m	2008-03-19	4.130	4.180	4.155
2m	2008-04-21	4.210	4.270	4.240
3m	2008-05-19	4.300	4.360	4.330
6m	2008-08-19	4.290	4.350	4.320
9m	2008-11-19	4.290	4.350	4.320
1y	2009-02-19	4.290	4.350	4.320

Table 1: Deposit rates. Instruments used in the bootstrap are highlighted in blue: maturity $\leq t_{\text{fut},1}^{\text{settle}}$ = 19th March 2008, corresponding to the overnight, 1-week and 1-month tenors.

2.2 Futures

Quotation	Settle	Expiry	BID	ASK	Mid	Fwd Rate (%)
17-Mar-08	19-Mar-08	19-Jun-08	95.680	95.685	95.683	4.318
16-Jun-08	18-Jun-08	18-Sep-08	96.065	96.070	96.068	3.933
15-Sep-08	17-Sep-08	17-Dec-08	96.370	96.375	96.373	3.628
15-Dec-08	17-Dec-08	17-Mar-09	96.525	96.530	96.528	3.473
16-Mar-09	18-Mar-09	18-Jun-09	96.605	96.610	96.608	3.393
15-Jun-09	17-Jun-09	17-Sep-09	96.570	96.580	96.575	3.425
14-Sep-09	16-Sep-09	16-Dec-09	96.490	96.495	96.493	3.508
14-Dec-09	16-Dec-09	16-Mar-10	96.365	96.375	96.370	3.630
15-Mar-10	17-Mar-10	17-Jun-10	96.275	96.285	96.280	3.720

Table 2: Futures prices and implied forward rates. Instruments used in the bootstrap are highlighted in blue: expiry $\leq t_{2y}^{\text{mat}}$ = 19th February 2010 (the 2y-swap settlement).

2.3 Swaps

Tenor	Maturity	BID (%)	ASK (%)
1y	2009-02-19	4.120	4.140
2y	2010-02-19	3.861	3.882
3y	2011-02-21	3.882	3.902
4y	2012-02-20	3.948	3.968
5y	2013-02-19	4.032	4.052
⋮	⋮	⋮	⋮
50y	2058-02-19	4.647	4.687

Table 3: Swap rates. Instruments used in the bootstrap are highlighted in blue: maturity $\geq t_{2y}^{\text{mat}}$. In particular the 1y swap is excluded as futures already cover that segment of the curve.

3 Methodology

Implementation Note

Throughout the bootstrap procedure, the maturity dates and the associated discount factors are progressively stored in two lists, `termDates` and `discounts`, which are initialized as:

$$\texttt{termDates} = [t_0], \quad \texttt{discounts} = [B(t_0, t_0)] = [1] \tag{1}$$

At each step, the newly bootstrapped discount factor and its maturity date are appended to these lists, which serve as the running state of the curve throughout the function.

All year fractions and discount factors are computed with respect to t_0. As requested we add to our vectors only the end dates of underlying contracts and their relative discounts.

3.1 Functions Used

The bootstrap is implemented in `ex0_utilities.py` through three functions:

- `from_discount_factors_to_zero_rates(dates, discount_factors)`: converts a set of discount factors into continuously compounded zero rates via $z(t) = -\log B(t_0, t)/t$. It accepts either a list of year fractions or a `DatetimeIndex` as input, handling the conversion

automatically. To avoid division by zero, the function automatically discards the first element corresponding to the spot date t_0, since $B(t_0, t_0) = 1$ and the resulting zero rate $z(t_0)$ is mathematically undefined.

- `get_discount_factor_by_zero_rates_linear_interp(reference_date, interp_date, dates, discount_factors)`: returns the discount factor at an arbitrary intermediate date t by performing linear interpolation on zero rates:

$$z(t) = z(t_{i-1}) + \frac{t - t_{i-1}}{t_i - t_{i-1}} \cdot [z(t_i) - z(t_{i-1})] \tag{2}$$

  The discount factor is then recovered via continuous compounding:

$$B(t_0, t) = e^{-z(t) \cdot t} \tag{3}$$

  The function also applies flat extrapolation beyond the last available pillar.

- `bootstrap(reference_date, depo, futures, swaps, shock)`: the main function. It sequentially bootstraps the discount curve from deposits, futures, and swaps, calling the two functions above when needed. It returns two `pd.Series`: the discount factors and the zero rates, indexed by maturity date.

3.2 Deposits

Deposits give directly the discount factor via the simple compounding formula:

$$B(t_0, t_i) = \frac{1}{1 + L(t_0, t_i) \cdot \delta(t_0, t_i)} \tag{4}$$

where $\delta(t_0, t_i)$ is the year fraction computed with ACT/360 convention.

We selected the deposits whose maturity satisfied $t_i \leq t_{\text{fut},1}^{\text{settle}}$, where $t_{\text{fut},1}^{\text{settle}}$ is the settlement date of the first future, computed as the first futures quotation date shifted by +2 business days. In this specific case, $t_{\text{fut},1}^{\text{settle}} = 19$ March 2008, and one deposit matured exactly on that date; therefore the first three deposits were included in the bootstrapping, up to and including the 19th of March 2008.

3.3 Futures

Futures provide forward rates $L(t_0; t_{i-1}, t_i)$. The discount factor is obtained via:

$$B(t_0, t_i) = B(t_0, t_{i-1}) \cdot \frac{1}{1 + L(t_0; t_{i-1}, t_i) \cdot \delta(t_{i-1}, t_i)} \tag{5}$$

where $\delta(t_{i-1}, t_i)$ is the year fraction computed with ACT/360 convention. We included futures whose expiry satisfied $t_i^{\text{expiry}} \leq t_{2y}^{\text{mat}}$, where t_{2y}^{mat} is the maturity of the 2y swap. The forward rate for each future is obtained as:

$$L(t_0; t_{i-1}, t_i) = \frac{100 - P_i}{100} \tag{6}$$

where P_i is the mid price, computed as the average of bid and ask quotes.

The discount factor $B(t_0, t_{i-1})$ at the settlement date of each future is retrieved from the already bootstrapped curve. If the settlement date does not coincide with an already bootstrapped pillar, linear interpolation on zero rates is performed. The flat extrapolation is used when needed.

3.4 Swaps

The long end of the curve is bootstrapped using interest rate swaps. From the par swap condition:

$$1 = R \sum_{i=1}^{N} \delta_i \cdot B(t_0, T_i) + B(t_0, T_N) \tag{7}$$

we can solve for the unknown discount factor at maturity $B(t_0, T_N)$:

$$B(t_0, T_N) = \frac{1 - R \sum_{i=1}^{N-1} \delta_i \cdot B(t_0, T_i)}{1 + R \cdot \delta_N} \tag{8}$$

where R is the swap rate and $\delta_i = \delta(T_{i-1}, T_i)$ is the year fraction computed with the 30E/360 convention.

The procedure is iterative: at each swap maturity T_N, all intermediate discount factors $B(t_0, T_i)$ for previous coupon dates ($i < N$) must be known. If an intermediate coupon date coincides with an already bootstrapped pillar, its discount factor is used directly. If the date is missing, it is retrieved via linear interpolation on zero rates, using flat extrapolation when needed.

In our dataset, the swap bootstrap begins directly from the 2y swap. For this first swap, the intermediate 1y coupon might not perfectly match previous futures settlements, triggering the interpolation. For all subsequent maturities (3y, 4y, ...), intermediate pillars are already available from previous iterations, as swaps pay annual coupons that align with previously bootstrapped maturities.

4 Results

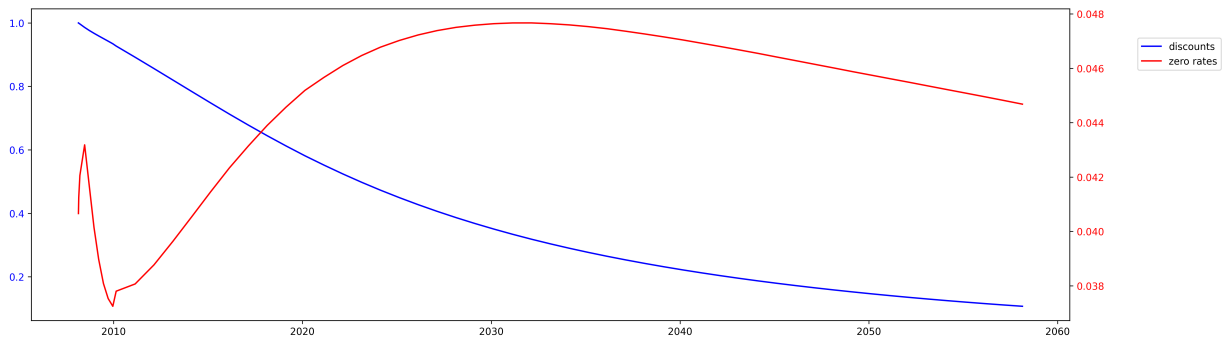

Figure 1: Bootstrapped discount factors and zero rates as of 15th February 2008.

The discount factor curve decreases monotonically from 1 at t_0 to approximately 0.15 at the 50y horizon, strictly bounded between 0 and 1 as mathematically expected. The zero rate curve, on the other hand, displays a non-monotonic shape, reflecting the specific market conditions of early 2008.

At the short end, zero rates are relatively high (around 4.3%) due to short-term market tensions. Moving towards the 2–3 year maturities, the curve exhibits a steep drop, reaching a local minimum of approximately 3.8%. This sharp decline indicates that the market was anticipating an imminent reduction in interest rates to counter a macroeconomic slowdown.

Beyond the 3-year pillar, the zero rate curve recovers and rises monotonically, stabilizing around 4.8% at the 25y tenor. Finally, in the ultra-long end (beyond 25y), the rates show a slight decline,

which is a typical structural feature of long-dated swap markets.

5 Limitations and Critical Aspects

Instrument Selection and Boundary Conditions

According to the assignment instructions, we separated the instruments using fixed dates: deposits up to the first future settlement, and futures up to the 2-year swap maturity.

A theoretical issue with this fixed-date rule is the transition between deposits and futures. By not using any deposit that matures *after* the first future settlement, we could create a time gap in the data. If this gap exists, computing the discount factor $B(t_0, t_{\text{settle}})$ for the first future requires **extrapolating** from much shorter deposits (like overnight rates). Mathematically, extrapolation is less stable than interpolation. Keeping a longer overlapping deposit would be a safer choice because it allows for standard linear interpolation. Fortunately, in our specific dataset the dates align perfectly, so this theoretical limitation does not cause practical problems.

6 Answer to Q1

> *"Bootstrap is not the only technique to obtain Discount Factors (DFs) from quoted rates. Why is it so relevant to implement the bootstrap of DFs in finance?"*

Bootstrap is not the only technique for extracting discount factors from market data, but it is by far the dominant one in practice for several reasons:

1. **Exact replication of market quotes:** The bootstrap algorithm ensures that the discount factors are calibrated so that the theoretical price of each instrument used in the construction coincides exactly with its market quotation. This is achieved by solving, at each maturity, the closed-form formulas associated with depos, futures, and swaps, described above in the "Methodology" section. As a consequence, the resulting curve is, by construction, fully consistent with the market data.
2. **Consistency with cash flow structure:** The method solves discount factors sequentially, using instruments whose cash flows depend only on discount factors already determined at earlier maturities. This guarantees no arbitrage across maturities, internal consistency, and a curve that faithfully reflects the term structure implied by deposits, STIR futures, and swaps.
3. **Integration of heterogeneous instruments:** Deposits, futures, and swaps differ in day count conventions, settlement rules, compounding conventions, payment frequencies, and maturities. The bootstrap method stitches all of them together into a single, coherent discount curve.
4. **Production of discount factors for valuation:** Market quotes are not directly usable for pricing. The discount factors given by the bootstrap are the fundamental building blocks for present value calculations, forward rate extraction, risk sensitivities, and scenario analysis.

To sum up, the bootstrap method is **exact** (it reproduces market prices), **consistent** (it enforces no arbitrage across maturities), **flexible** (it integrates deposits, futures, and swaps into a unified curve), and **fundamental** (it produces the discount factors required by pricing and risk management models).

7 Answer to Q2

"Can you specify which are the differences with the Bootstrap you made in MFII?"

The current Python implementation significantly differs from the bootstrap methodology developed in MATLAB for the MF2 course. The main distinctions can be summarized as follows:

- **Set of Instruments:** The two bootstraps rely on different sets of market instruments. In our current case, we use short-term deposits (up to 1 month), the first seven highly liquid STIR futures, and medium-to-long-term IRS swaps. Conversely, the MF2 bootstrap relied on one Eurodollar future, two FRA contracts, and five OIS swaps.
- **Solution Methodology:** The Python bootstrap utilizes closed-form formulas for each instrument: deposits yield a direct discount factor, futures provide a forward discount factor, and swaps yield the final discount factor via the par swap condition. The algorithm is strictly deterministic and requires no root-finding. In contrast, the MATLAB bootstrap employs numerical root-finding: each instrument defines a pricing equation where the unknown is the zero rate at the new pillar, and the equation is solved using the `fzero` function.
- **Role of Interpolation:** In the Python code, interpolation is applied only when strictly necessary (e.g., when a futures settlement date or a swap coupon date does not coincide with an existing pillar), and it is always linear on zero rates. In the MATLAB code, interpolation is an integral part of the pricing model itself. The user can choose between linear, spline, or pchip methods, and this choice directly affects every pricing equation solved by `fzero`. Consequently, the entire curve and the resulting forward rates are highly sensitive to the chosen interpolation method.
- **Market Conventions:** The Python implementation adopts highly realistic market conventions: ACT/360 for deposits and futures, 30E/360 for swaps, ACT/365 for zero rates, T+2 settlement, and precise futures expiry rules (settlement + 3 months). This produces a curve that accurately reflects how real instruments are quoted and traded. The MATLAB bootstrap, on the other hand, simplifies the implementation by applying a single day-count convention (ACT/360) across all instruments, which is less realistic.
- **Shock Mechanism:** Finally, the Python bootstrap includes a built-in mechanism to apply shocks to the input rates, allowing for both parallel shifts and pillar-specific adjustments. The MATLAB bootstrap does not feature any mechanism for sensitivity or scenario analysis.